

Assignment — “Mini-Gateway Fraud & OCR Prototype”

Scenario

AajPay is piloting a new micro-merchant onboarding flow. For each sign-up we capture:

1. **Transaction snapshot** – core features (amount, device_id, geo, BIN, etc.).
2. **Receipt image** – a photo of the first transaction’s printed slip.

Risk wants an MVP that (a) flags likely-fraudulent sign-ups **in real-time**, and (b) extracts the merchant name & total from the receipt image to feed a rules engine.

Your Task (deliver all items)

Part	What you build	Success criteria
A. Data EDA + Baseline Model	• Explore the CSV (<5 MB, class-imbalance $\approx 1 : 800$). • Create a fraud-probability model. • Save as <code>model.pkl</code>.	≥ 0.75 PR-AUC on the hidden test set (we’ll rerun).
B. OCR Pipeline	• For each receipt (<code>/images/*.jpg</code>), detect the printed area, deskew, run OCR, and parse merchant_name & total_amount. • Produce <code>ocr_results.json</code>.	$\leq 5\%$ key-field error rate on our grader.
C. REST Inference Service	• FastAPI endpoint <code>/score</code> that accepts JSON <code>{"transaction": {...}, "receipt_path": "..."}.</code> • Returns <code>{"fraud_score": 0-1, "merchant_name": "...", "total": ...}.</code> • Package in Docker.	p99 latency < 60 ms on a single request (CPU).
D. README + Report	• How you handled imbalance, features created, OCR choices. • Instructions to run Docker image & reproduce metrics.	Clear, step-by-step; we can stand-up the service in ≤ 10 min.

(A small, anonymised dataset + image bundle is provided in the drive link. If you need synthetic data, you may generate but document the process.)

Submission Guidelines

Item	Format
Code & notebooks	Push to a private Git repo or zip file.
Docker image	Dockerfile + build steps, or push to Docker Hub/GHCR.
Report	Markdown or PDF (≤ 3 pages).
Deadline	72 hours after you receive the dataset link.

How We'll Evaluate

1. **Model quality** — PR-AUC & recall at 0.5 % FPR.
 2. **Latency & resource use** — we'll hit the container with 100 req/s burst.
 3. **Code clarity & tests** — readability, logging, minimal Makefile or CI script.
 4. **Sound reasoning** — feature choices, imbalance strategy, OCR error handling.
 5. **Bonus points** — concept-drift detector stub, or ONNX-/TensorRT optimisation.
-

After Submission

- We'll run an automated grader and skim your code.
- You'll then join a **30 min technical debrief** to walk us through decisions and trade-offs.